

Risk-Based Automated QA for a TypeScript Full-Stack Monorepo

Aldiyar Seyl Khanov, Alexandr Tyul'kov, Malika Ishakhanova

Astana IT University | Software QA and Testing | Circles case study



Introduction

Circles is a TypeScript monorepo for Spotify listening-history analytics. It contains a React frontend, a Hono backend, shared packages, PostgreSQL storage, and background import workflows.

The QA problem is not only whether tests pass. The main problem is whether high-risk behavior is covered by the right evidence. This is important because TypeScript projects can still fail through toolchain and configuration boundaries, not only through application logic [1].

33/33 Backend tests extended suite passed	2 Workflows GitHub Actions checks
16,600 Dataset synthetic history rows	Vite+ Toolchain one command surface

Main risk areas: authentication, Spotify export ingestion, dashboard range filtering, API routing, CI test composition, and PostgreSQL availability.

Literature Evidence

The poster keeps the evidence base from the research paper. Risk-based testing supports prioritizing high-impact failure areas instead of distributing effort evenly [2], [3].

Continuous integration literature supports frequent automated checks, but also shows that pipeline cost, configuration, and test stability matter [4], [5].

Coverage is used as a signal, not as proof of quality. Prior work shows that coverage alone is weakly related to test effectiveness when suite size is controlled [6]. Mutation testing is therefore used for selected helper modules because it gives a stronger adequacy signal [7].

Research Question

Research question. How can risk-based planning, automation, quality gates, and experiments be connected into one reproducible QA argument for Circles?

Method. Earlier assignments were integrated into one pipeline. Assignment 1 supplied the risk strategy. Assignment 2 supplied automated tests and gates. Assignment 3 supplied performance, mutation, and chaos evidence. The final paper connected those parts into one case study.

Claim. The current QA process is useful but incomplete. It gives fast feedback and exposes real weaknesses, but the strongest evidence is concentrated in helper modules rather than full user workflows.

Methodology: Risk Model

Risks were scored with a simple model:

$$\text{Risk Score} = \text{Likelihood} \times \text{Impact}$$

Risk	Area	Score	Evidence
R1	Sign-in and sessions	20	Partial
R2	Spotify export ingestion	20	Strong helper tests
R3	Range-filtered analytics	15	Unit + load
R4	API and middleware contracts	12	Gap
R5	Frontend test composition	12	Partial
R6	Database outage behavior	15	Chaos test

The model changed test selection. High-impact paths were not treated as equal to low-risk helper code. Performance and resilience experiments were kept separate because load behavior and failure recovery need environment-level evidence [8], [9].

Methodology: QA Pipeline

1. Risks	2. Tests	3. Gates	4. Experiments	5. Review
likelihood and impact scoring	unit, integration, E2E, mutation	pass, coverage, latency, mutation	load and failure injection	limitations and next tests

Tool	Role	Reason
Vite+	Unified command interface	Matches repository workflow
Vitest	Unit and component tests	Fast TypeScript feedback
Stryker	Mutation testing	Checks assertion strength
Autocannon	Performance testing	Repeatable local HTTP load
GitHub Actions	CI enforcement	Push and pull-request checks

Results

87.93% Mutation score selected helper files	24 ms Worst P99 local read endpoint
< 7% Backend coverage whole-backend gate failed	200 again Recovery after PostgreSQL restart

Selected helper mutation results

range.ts	100
retry.ts	77
spotify-export.ts	100
zip.ts	88

The load test used 20 concurrent connections for 10 seconds. The worst observed P99 was 24 ms on /leaderboard?period=all. The chaos test followed a 200, 500, 200 sequence when PostgreSQL was available, stopped, and restarted.

Discussion: Key Findings

The results show a useful but incomplete QA process. The extended backend suite passed with 33 tests, and selected helper modules reached an 87.93% mutation score. Local read performance was also strong, with the worst observed P99 at 24 ms. These results support the value of the risk-based automation plan.

The same evidence also shows clear gaps. Whole-backend coverage stayed below 7%, so the coverage gate failed at system scope. The PostgreSQL chaos experiment produced a user-visible HTTP 500 during the outage, although the service recovered after the database restarted. The strongest evidence is therefore concentrated in helper modules, while authentication, route composition, and database-backed workflows still need broader integration tests.

Conclusion

The next iteration should expand breadth before adding more tools.

- Add integration tests for authenticated backend controllers.
- Test route middleware and public/private route boundaries.
- Strengthen Spotify import validation for malformed exports.
- Repeat performance tests with larger data and multiple runs.
- Add graceful error handling for database outages.
- Keep mutation testing focused on high-risk modules unless runtime cost is reduced.

The main defense point is that passing tests are not enough. Evidence must show which risks are covered and which risks remain open.

Limitations

The study has four limits:

- It uses one repository, so the result is a case study.
- Performance data comes from a local machine.
- Mutation testing covers four helper files only.
- Controller and workflow integration coverage is still incomplete.

References and Evidence



Repository source

Scan for the code, tests, and QA artifacts.
github.com/anyrange/circles

Cited sources

- [1] T. Terp, S. Almadadi, and N. Sumari, "From Logic to Toolchains: An Empirical Study of Bugs in the TypeScript Ecosystem," in *Proceedings of the 23rd International Conference on Mining Software Repositories*, ACM, 2020, pp. 1–11, doi: 10.1145/3733302.3733379.
- [2] B. W. Boehm, "Software Risk Management: Principles and Practices," *IEEE Software*, vol. 8, no. 1, pp. 32–41, 1991, doi: 10.1109/52.62930.
- [3] M. Felderer and I. Schieferdecker, "A Taxonomy of Risk-Based Testing," *Software Quality: Model-Based Approaches for Advanced Software and Systems Engineering*, Springer, pp. 121–136, 2014, doi: 10.1007/978-3-319-03602-1_8.
- [4] Y. Wang, M. V. Adeniyi, Z. Liu, and J. Markkula, "Test Automation Maturity Improves Product Quality: Quantitative Study of Open Source Projects Using Continuous Integration," *Journal of Systems and Software*, vol. 188, p. 111259, 2022, doi: 10.1016/j.jss.2022.111259.
- [5] M. Shahin, M. A. Babar, and L. Zhu, "Continuous Integration, Delivery and Deployment: A Systematic Review on Approaches, Tools, Challenges and Practices," *IEEE Access*, vol. 5, pp. 3909–3943, 2017, doi: 10.1109/ACCESS.2017.2686629.
- [6] L. Inozentseva and R. Holmes, "Coverage is Not Strongly Correlated with Test Suite Effectiveness," *Proceedings of the 36th International Conference on Software Engineering*, pp. 435–445, 2014, doi: 10.1145/2588225.2588271.
- [7] Y. Ju and M. Harman, "An Analysis and Survey of the Development of Mutation Testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 949–978, 2011, doi: 10.1109/TSE.2010.102.
- [8] Z. M. Jiang and A. E. Hassan, "A Survey on Load Testing of Large-Scale Software Systems," *IEEE Transactions on Software Engineering*, vol. 41, no. 11, pp. 1091–1118, 2016, doi: 10.1109/TSE.2015.2445340.
- [9] A. Basiri et al., "Chaos Engineering," in *IEEE International Symposium on Software Reliability Engineering Workshops*, IEEE, 2016, pp. 1–6, doi: 10.1109/ISSREW.2016.21.